

# Spring Boot et Thymeleaf

Application pas à pas

## 1 QU'EST-CE QUE THYMELEAF ?

---

Thymeleaf est un moteur de templates (Template Engine) Java, open source sous licence Apache 2.0. Il fonctionne dans des environnements Web (Servlet) comme dans des environnements non Web, et s'intègre complètement à Spring Framework.

Il est surtout utilisé pour produire du HTML5 dans la couche Vue (View Layer) d'une application Web construite selon le modèle MVC. Il sait aussi traiter du XML, du texte, du JavaScript ou du CSS (voir le document d'introduction).

Le moteur lit un fichier de template, le combine avec des objets Java (le modèle) et génère un autre document. Thymeleaf est la solution recommandée pour remplacer JSP dans une application Spring Boot : JSP est peu adapté aux JAR exécutables que Spring Boot produit par défaut, alors que Thymeleaf fonctionne sans configuration particulière.

Dans ce TP, nous réalisons une petite application de gestion de personnes, avec trois pages :

- Une page d'accueil ;
- La liste des personnes ;
- Un formulaire d'ajout d'une personne, avec validation des champs.

## 2 CRÉER L'APPLICATION

Nous créons le projet à partir de **Spring Initializr**, intégré à IntelliJ IDEA (*File → New → Project → Spring Boot*, édition Ultimate) ou accessible en ligne sur [start.spring.io](https://start.spring.io) (compatible avec toutes les éditions d'IntelliJ et avec les autres IDE).

Saisissez les paramètres suivants :

| Paramètre    | Valeur                                                  |
|--------------|---------------------------------------------------------|
| Project      | Maven                                                   |
| Language     | Java                                                    |
| Spring Boot  | la dernière version stable proposée (3.x ou supérieure) |
| Name         | <i>demoSBT</i>                                          |
| Group        | <i>local.epul4a</i>                                     |
| Artifact     | <i>demosbt</i>                                          |
| Description  | Spring Boot and Thymeleaf                               |
| Package name | <i>local.epul4a.demosbt</i>                             |
| Packaging    | Jar                                                     |
| Java         | 17 ou supérieur                                         |

Ajoutez les dépendances suivantes (champ *Dependencies*) :

- **Spring Web** : Spring MVC et serveur Tomcat embarqué ;
- **Thymeleaf** : le moteur de templates ;
- **Validation** : Bean Validation (*@NotBlank*, *@Valid*...), utilisé pour le formulaire ;
- **Spring Boot DevTools** (facultatif) : rechargement automatique pendant le développement.

**Remarque.** Le nom de l'artefact Maven s'écrit par convention en minuscules, d'où *demosbt*.  
Les noms de paquets Java sont eux aussi toujours en minuscules et sans espace.

Le projet généré a la structure suivante. Nous allons ajouter les fichiers marqués d'une étoile :

```
demoSBT
├── pom.xml
├── src
│   ├── main
│   │   ├── java/local/epul4a/demosbt
│   │   │   ├── DemosbtApplication.java
│   │   │   ├── controller/MainController.java *
│   │   │   ├── form/PersonForm.java *
│   │   │   └── model/Person.java *
│   │   └── resources
│   │       ├── application.properties
│   │       ├── static/css/style.css *
│   │       └── templates
│   │           ├── index.html *
│   │           └── personList.html *
```

```

└─ test/java/local/epul4a/demosbt
    └─ controller/MainControllerTest.java *

```

### 3 LES TEMPLATES THYMELEAF

---

Un template Thymeleaf est un fichier HTML5. Nous créons trois fichiers dans `src/main/resources/templates` (c'est le répertoire où Spring Boot cherche les templates par défaut).

#### 3.1 `index.html`

```

<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Bienvenue</title>
  <link rel="stylesheet" href="../static/css/style.css"
        th:href="@{/css/style.css}" />
</head>
<body>
  <h1>Bienvenue</h1>

  <!--/*@thymesVar id="message" type="java.lang.String"*/-->
  <h2 th:text="${message}">Message d'accueil</h2>

  <a th:href="@{/personList}">Lister Les personnes</a>
</body>
</html>

```

Le commentaire `<!--/*@thymesVar ... */-->` est une **indication pour l'IDE** (IntelliJ) : il déclare le type de la variable `message` afin d'activer l'autocomplétion. Comme tout commentaire de la forme `<!--/* ... */-->`, il est supprimé du HTML produit.

#### 3.2 `PERSONLIST.HTML`

```

<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Liste des personnes</title>
  <link rel="stylesheet" href="../static/css/style.css"
        th:href="@{/css/style.css}" />
</head>
<body>
  <h1>Liste des personnes</h1>

  <a th:href="@{/addPerson}">Ajouter une personne</a>
  <br/><br/>

  <p th:if="${#lists.isEmpty(persons)}">Aucune personne enregistrée.</p>

```

```

<table th:unless="{#lists.isEmpty(persons)}">
  <tr>
    <th>Prénom</th>
    <th>Nom</th>
  </tr>
  <tr th:each="person : {persons}">
    <td th:text="{person.firstName}">Prénom</td>
    <td th:text="{person.lastName}">Nom</td>
  </tr>
</table>
</body>
</html>

```

### 3.3 3.3 ADDPERSON.HTML

```

<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Ajouter une personne</title>
  <link rel="stylesheet" href="../static/css/style.css"
        th:href="@{/css/style.css}" />
</head>
<body>
  <h1>Créer une personne</h1>

  <form th:action="@{/addPerson}" th:object="{personForm}" method="post">

    <Label for="firstName">Prénom :</Label>
    <input type="text" th:field="*{firstName}" />
    <span class="error" th:if="{#fields.hasErrors('firstName')}}"
      th:errors="*{firstName}">Erreur</span>
    <br/>

    <Label for="lastName">Nom :</Label>
    <input type="text" th:field="*{lastName}" />
    <span class="error" th:if="{#fields.hasErrors('lastName')}}"
      th:errors="*{lastName}">Erreur</span>
    <br/>

    <input type="submit" value="Créer" />
  </form>

  <p><a th:href="@{/personList}">Retour à la Liste</a></p>
</body>
</html>

```

Quelques points importants :

- `th:object="{personForm}"` associe le formulaire à un objet du modèle, et `th:field="*{firstName}"` génère automatiquement les attributs `id`, `name` et `value` du champ (ce qui permet au `<Label for="firstName">` de fonctionner).
- `#fields.hasErrors('firstName')` et `th:errors` affichent les erreurs de validation du champ.

- `th:action="@{/addPerson}"` ajoute automatiquement le chemin de contexte, et, si Spring Security est utilisé, le jeton **CSRF** dans le formulaire.
- Nous utilisons `th:text` et non `th:utext` : il échappe le HTML saisi par l'utilisateur et protège contre les failles XSS.

### 3.4 3.4 À PROPOS DU BALISAGE

Tous les fichiers déclarent l'espace de noms Thymeleaf :

```
<html xmlns:th="http://www.thymeleaf.org">
```

Thymeleaf traite les attributs `th:*` même sans cette déclaration, mais elle est utile pour l'IDE (autocomplétion, validation) et pour la lisibilité. En mode HTML, Thymeleaf est tolérant : le document n'a pas à être du XML bien formé. Il reste cependant recommandé de fermer toutes les balises (`<br />`, `<meta ... />`) pour garder des templates propres.

Les attributs `th:href` et `th:src` remplacent l'attribut `href / src` classique lors du traitement. Les valeurs `href=" ../static/css/style.css "` écrites à côté ne servent que lorsqu'on ouvre le fichier directement dans un navigateur (*Natural Template*).

### 3.5 3.5 LE CHEMIN DE CONTEXTE (CONTEXT PATH)

Les expressions d'URL `@{...}` ajoutent automatiquement le chemin de contexte de l'application. Il se définit dans `application.properties` :

```
server.servlet.context-path=/my-context-path
```

```
<!-- Exemple 1 -->
<a th:href="@{/mypath/abc.html}">Un Lien</a>
<!-- Résultat : -->
<a href="/my-context-path/mypath/abc.html">Un Lien</a>

<!-- Exemple 2 -->
<form th:action="@{/mypath/abc.html}" method="post">
<!-- Résultat : -->
<form action="/my-context-path/mypath/abc.html" method="post">
```

Sans cette propriété, le chemin de contexte est vide (/) et `@{/personList}` produit `/personList`.

## 4 4. RESSOURCES STATIQUES ET FICHIER DE PROPRIÉTÉS

Les ressources statiques (CSS, JavaScript, images...) se placent dans `src/main/resources/static` ou l'un de ses sous-dossiers. Le fichier `static/css/style.css` est accessible à l'URL `/css/style.css`.

### 4.1 4.1 STYLE.CSS

```
h1 { color: #0000FF; }
h2 { color: #FF0000; }
```

```
table { border-collapse: collapse; }
table, th, td { border: 1px solid black; }
table th, table td { padding: 5px; }

.error { color: red; font-style: italic; }
```

## 4.2 4.2 APPLICATION.PROPERTIES

```
spring.application.name=demoSBT

# Message d'accueil, injecté dans le contrôleur
welcome.message=Demo Thymeleaf

# Désactive le cache des templates pendant le développement
spring.thymeleaf.cache=false
```

**Remarque.** `spring.thymeleaf.cache=false` permet de voir immédiatement les modifications de templates. Avec **Spring Boot DevTools**, cette propriété est déjà désactivée automatiquement en développement ; en production, on laisse le cache activé (valeur par défaut).

# 5 5. LES CLASSES MODEL, FORM ET CONTROLLER

---

## 5.1 5.1 PERSON.JAVA

```
package Local.epul4a.demosbt.model;

public class Person {

    private final String firstName;
    private final String lastName;

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
}
```

## 5.2 5.2 LA CLASSE PERSONFORM

Elle représente les données du formulaire de création d'une personne (page [addPerson](#)). On y déclare les règles de validation avec **Bean Validation**. Séparer le *form* du *model* évite de lier la vue directement à l'objet métier.

```
package Local.epul4a.demosbt.form;

import jakarta.validation.constraints.NotBlank;

public class PersonForm {

    @NotBlank(message = "Le prénom est obligatoire")
    private String firstName;

    @NotBlank(message = "Le nom est obligatoire")
    private String lastName;

    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }

    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
}
```

`@NotBlank` refuse `null`, la chaîne vide et les chaînes composées uniquement d'espaces. Depuis Spring Boot 3, les annotations de validation appartiennent au paquet `jakarta.validation` (et non plus `javax.validation`).

### 5.3 LA CLASSE `MAINCONTROLLER`

`MainController` est le contrôleur : il traite les requêtes de l'utilisateur et pilote le flux de l'application.

```
package Local.epul4a.demosbt.controller;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

import jakarta.validation.Valid;
import Local.epul4a.demosbt.form.PersonForm;
import Local.epul4a.demosbt.model.Person;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class MainController {

    // Données en mémoire (démonstration). Le contrôleur est un singleton
    // partagé entre les requêtes : on utilise donc une liste thread-safe.
    private final List<Person> persons = new CopyOnWriteArrayList<>();
}
```

```

private final String message;

// Injection par constructeur de la valeur définie dans application.properties
public MainController(@Value("${welcome.message}") String message) {
    this.message = message;
    persons.add(new Person("Bill", "Gates"));
    persons.add(new Person("Steve", "Jobs"));
}

@GetMapping("/{", "/index"})
public String index(Model model) {
    model.addAttribute("message", message);
    return "index";
}

@GetMapping("/personList")
public String personList(Model model) {
    model.addAttribute("persons", persons);
    return "personList";
}

@GetMapping("/addPerson")
public String showAddPersonPage(Model model) {
    model.addAttribute("personForm", new PersonForm());
    return "addPerson";
}

@PostMapping("/addPerson")
public String savePerson(@Valid @ModelAttribute("personForm") PersonForm personForm,
    BindingResult result) {
    if (result.hasErrors()) {
        return "addPerson"; // on réaffiche le formulaire avec les erreurs
    }
    persons.add(new Person(personForm.getFirstName(), personForm.getLastName()));
    return "redirect:/personList"; // Post/Redirect/Get
}
}

```

Explications :

- `@GetMapping` et `@PostMapping` sont les formes abrégées de `@RequestMapping(method = GET/POST)`.
- `@Valid` déclenche la validation de `PersonForm`. Les erreurs sont placées dans `BindingResult`, qui doit se trouver **juste après** l'objet validé dans la signature de la méthode.
- Après un ajout réussi, on **redirige** (`redirect:/personList`) au lieu d'afficher directement la liste : c'est le motif `Post/Redirect/Get`, qui évite qu'un rafraîchissement de la page (F5) renvoie le formulaire une seconde fois.

- Les personnes sont stockées en mémoire, et sont donc perdues à l'arrêt de l'application. Dans une vraie application, on utiliserait un service et une base de données (Spring Data JPA).

## 6 EXÉCUTER L'APPLICATION

Lancez la classe *DemosbtApplication* depuis l'IDE, ou en ligne de commande à la racine du projet :

```
./mvnw spring-boot:run
```

Ouvrez ensuite les URL suivantes dans un navigateur :

| URL                                                                             | Résultat attendu                                                         |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <a href="http://localhost:8080/">http://localhost:8080/</a>                     | Page d'accueil : le titre « Bienvenue » et le message « Demo Thymeleaf » |
| <a href="http://localhost:8080/personList">http://localhost:8080/personList</a> | Tableau avec Bill Gates et Steve Jobs                                    |
| <a href="http://localhost:8080/addPerson">http://localhost:8080/addPerson</a>   | Formulaire de création                                                   |

Essayez de valider le formulaire avec un champ vide : la page doit se réafficher avec le message d'erreur sous le champ concerné. Avec des champs remplis, vous êtes redirigé vers la liste, qui contient la nouvelle personne.

## 7. TESTER L'APPLICATION

Un test de tranche Web (*@WebMvcTest*) vérifie le contrôleur et le rendu des vues sans démarrer tout le serveur. Il utilise *MockMvc*, fourni par le starter de test déjà présent dans le projet.

```
package Local.epul4a.demosbt.controller;

import static org.hamcrest.Matchers.containsString;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(MainController.class)
class MainControllerTest {

    @Autowired
    private MockMvc mvc;

    @Test
```

```
void personListShowsInitialPersons() throws Exception {
    mvc.perform(get("/personList"))
        .andExpect(status().isOk())
        .andExpect(content().string(containsString("Gates")));
}

@Test
void invalidFormStaysOnFormWithErrors() throws Exception {
    mvc.perform(post("/addPerson")
        .param("firstName", "")
        .param("lastName", "Turing"))
        .andExpect(status().isOk())
        .andExpect(view().name("addPerson"))
        .andExpect(model().attributeHasFieldErrors("personForm", "firstName"));
}

@Test
void validFormRedirectsToList() throws Exception {
    mvc.perform(post("/addPerson")
        .param("firstName", "Alan")
        .param("lastName", "Turing"))
        .andExpect(redirectedUrl("/personList"));
}
```

**Remarque.** Selon la version de Spring Boot, le paquet de l'annotation `@WebMvcTest` peut différer (il a été réorganisé dans les versions les plus récentes) : laissez l'IDE proposer l'import adapté à votre version.

## 8 RÉSUMÉ

---

- Un projet Spring Boot + Thymeleaf se crée avec Spring Initializr (dépendances Spring Web, Thymeleaf, Validation).
- Les templates se placent dans `src/main/resources/templates`, les ressources statiques dans `src/main/resources/static`.
- Les expressions `@{...}` produisent des URL qui tiennent compte du chemin de contexte.
- Le contrôleur remplit le modèle (*Model*) et retourne le nom du template ; un formulaire est lié à un objet par `th:object` et `th:field`.
- La validation se fait avec Bean Validation (`@NotBlank`, `@Valid`, `BindingResult`) et les erreurs s'affichent avec `th:errors`.
- On utilise `th:text` pour échapper le contenu, et le motif Post/Redirect/Get après un POST réussi.

## 9 EXERCICES PROPOSÉS

---

1. Ajouter à *Person* un champ *email* (avec *@Email* dans le formulaire) et l'afficher dans le tableau.
2. Ajouter un bouton « Supprimer » à chaque ligne de la liste (méthode *@PostMapping("/deletePerson")*), en expliquant pourquoi on évite d'utiliser un simple lien *GET* pour une suppression.
3. Extraire l'en-tête (menu de navigation) dans un fragment *fragments/header.html* réutilisé par les trois pages.
4. Remplacer la liste en mémoire par une entité JPA et un *JpaRepository* avec une base H2, puis écrire un test d'intégration.
5. Ajouter la prise en charge du français et de l'anglais avec des fichiers *messages.properties* / *messages\_en.properties* et l'expression *#{...}*.